

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Системы искусственного интеллекта
Лабораторная работа №2

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Королева Юлия Александровна

Содержание	
Текст задания.....	3
Часть 1.....	4
Алгоритмы.....	4
Реализация.....	6
Оценка моделей.....	20
Часть 2.....	23
Алгоритмы.....	23
Реализация.....	24
Оценка моделей.....	28
Вывод.....	29
Приложение.....	30

Текст задания

Задание лабораторной работы состоит из двух частей:

- Часть 1: Необходимо добавить в модели из лабораторной работы №1 регуляризацию L1, L2 и ElasticNet. Для каждой модели вывести графики ошибок/метрик. Использовать датасеты также из лабораторной работы №1;
- Часть 2: Используя Jupiter Notebook Реализовать класс модели KNN, обучить модель на датасетах для задач классификации и регрессии, вывести метрики качества.

Часть 1

Алгоритмы

Ниже представлены псевдокоды алгоритмов регуляризаций L1 (а), L2 (б) и ElasticNet (в).

$$a) z = X * w + b$$

$$y_{pred} = \text{sigmoid}(z)$$

$$error = y_{pred} - y$$

$$grad_w = \frac{1}{n} * X^T * error + \lambda * \text{sign}(w)$$

$$grad_b = \frac{1}{n} * \sum error$$

$$w = w - \alpha * grad_w$$

$$b = b - \alpha * grad_b$$

$$б) z = X * w + b$$

$$y_{pred} = \text{sigmoid}(z)$$

$$error = y_{pred} - y$$

$$grad_w = \frac{1}{n} * X^T * error + \lambda * w$$

$$grad_b = \frac{1}{n} * \sum error$$

$$w = w - \alpha * grad_w$$

$$b = b - \alpha * grad_b$$

$$в) z = X * w + b$$

$$y_{pred} = \text{sigmoid}(z)$$

$$error = y_{pred} - y$$

$$grad_w = \frac{1}{n} * X^T * error + \lambda * [r * \text{sign}(w) + (1 - r) * w]$$

$$grad_b = \frac{1}{n} * \sum error$$

$$w = w - \alpha * grad_w$$

$$b = b - \alpha * grad_w$$

Реализация

Далее на листингах 1 - 4 представлена реализация классов линейной и логистической регрессий с регуляризациями и их применение.

Листинг 1 - Реализация линейной регрессии с регуляризациями

```
@dataclass
class TrainingHistory:
    losses: list[float]
    train_loss_history: list[float]
    val_loss_history: list[float]
    train_mae_history: list[float]
    val_mae_history: list[float]
    train_rmse_history: list[float]
    val_rmse_history: list[float]
    weights_norm: list[float]

class LinearRegression:
    def __init__(self, learning_rate: float = 0.01, n_iterations: int = 10000, regularization: str = None,
                  tolerance: float = 1e-6, fit_intercept: bool = True,
                  verbose: bool = False, alpha: float = 0.01, l1_ratio: float = 0.5):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.tolerance = tolerance
        self.fit_intercept = fit_intercept
        self.verbose = verbose

        self.weights = None
        self.bias = 0.0 if fit_intercept else None
        self.is_fitted = False
        self.history = None

        self.regularization = regularization
        self.alpha = alpha
        self.l1_ratio = l1_ratio

    def compute_loss(self, y_true: torch.Tensor, y_pred: torch.Tensor,
                    weights: torch.Tensor) -> torch.Tensor:
        base_loss = torch.mean((y_true - y_pred) ** 2)

        reg = 0
        if self.regularization == 'l1':
            reg = self.alpha * torch.sum(torch.abs(weights))
```

```

        elif self.regularization == 'l2':
            reg = self.alpha * torch.sum(weights ** 2)
        elif self.regularization == 'elasticnet':
            l1_reg = self.alpha * self.l1_ratio *
torch.sum(torch.abs(weights))
            l2_reg = self.alpha * (1 - self.l1_ratio) *
torch.sum(weights ** 2)
            reg = l1_reg + l2_reg

        return base_loss + reg

    def compute_mae(self, y_true: torch.Tensor, y_pred: torch.Tensor) ->
torch.Tensor:
        return torch.mean(torch.abs(y_true - y_pred))

    def compute_rmse(self, y_true: torch.Tensor, y_pred: torch.Tensor)
-> torch.Tensor:
        return torch.sqrt(self.compute_loss(y_true, y_pred, self.weights))

    def compute_gradients(self, X: torch.Tensor, error: torch.Tensor,
weights: torch.Tensor) -> tuple:
        n_samples = X.shape[0]
        dw = (1 / n_samples) * torch.matmul(X.T, error)

        if self.regularization == 'l1':
            dw += self.alpha * torch.sign(weights)
        elif self.regularization == 'l2':
            dw += 2 * self.alpha * weights
        elif self.regularization == 'elasticnet':
            l1_grad = self.alpha * self.l1_ratio * torch.sign(weights)
            l2_grad = 2 * self.alpha * (1 - self.l1_ratio) * weights
            dw += l1_grad + l2_grad

        db = (1 / n_samples) * torch.sum(error) if self.fit_intercept
else 0

        return dw, db

    def gradient_descent(self, X: torch.Tensor, y: torch.Tensor, X_val:
torch.Tensor = None, y_val: torch.Tensor = None):
        n_samples, n_features = X.shape
        self.weights = torch.zeros(n_features, device=device)
        if self.fit_intercept:
            self.bias = torch.tensor(0.0, device=device)

        losses = []

```

```

train_loss_history = []
val_loss_history = []
train_mae_history = []
val_mae_history = []
train_rmse_history = []
val_rmse_history = []
weights_norm = []

for iteration in range(self.n_iterations):
    y_pred = torch.matmul(X, self.weights)
    if self.fit_intercept:
        y_pred += self.bias

    train_loss = self.compute_loss(y, y_pred, self.weights)
    train_mae = self.compute_mae(y, y_pred)
    train_rmse = self.compute_rmse(y, y_pred)

    train_loss_history.append(train_loss.item())
    train_mae_history.append(train_mae.item())
    train_rmse_history.append(train_rmse.item())
    losses.append(train_loss.item())
    weights_norm.append(torch.norm(self.weights).item())

    val_loss = val_mae = val_rmse = None
    if X_val is not None and y_val is not None:
        y_val_pred = torch.matmul(X_val, self.weights)
        if self.fit_intercept:
            y_val_pred += self.bias

        val_loss = self.compute_loss(y_val, y_val_pred,
self.weights)
        val_mae = self.compute_mae(y_val, y_val_pred)
        val_rmse = self.compute_rmse(y_val, y_val_pred)

        val_loss_history.append(val_loss.item())
        val_mae_history.append(val_mae.item())
        val_rmse_history.append(val_rmse.item())

    error = y_pred - y
    dw, db = self.compute_gradients(X, error, self.weights)

    self.weights -= self.learning_rate * dw

    if self.fit_intercept:
        self.bias -= self.learning_rate * db

```



```

        # if iteration > 0 and abs(losses[-2] - losses[-1]) <
self.tolerance:
        #     if self.verbose:
        #         print(f"Converged at iteration {iteration}")
        #     break

        if self.verbose and iteration % 500 == 0:
            msg = f"Iteration {iteration}, Train Loss:
{train_loss:.6f}"
            if val_loss is not None and val_mae is not None and
val_rmse is not None:
                msg += f", Val Loss: {val_loss:.6f}, MAE:
{val_mae:.6f}, RMSE: {val_rmse:.6f}"
            print(msg)

self.history = TrainingHistory(
    losses=losses,
    train_loss_history=train_loss_history,
    val_loss_history=val_loss_history,
    train_mae_history=train_mae_history,
    val_mae_history=val_mae_history,
    train_rmse_history=train_rmse_history,
    val_rmse_history=val_rmse_history,
    weights_norm=weights_norm
)

def fit(self, X: torch.Tensor, y: torch.Tensor, X_val: torch.Tensor
= None, y_val: torch.Tensor = None):
    self.gradient_descent(X, y, X_val, y_val)
    self.is_fitted = True

def predict(self, X: torch.Tensor) -> torch.Tensor:
    if not self.is_fitted:
        return X

    if X.ndim == 1:
        X = X.reshape(-1, 1)

    y_pred = torch.matmul(X, self.weights)
    if self.fit_intercept:
        y_pred += self.bias

    return y_pred

def score(self, X: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    y_pred = self.predict(X)

```

```

        ss_res = torch.sum((y - y_pred) ** 2)
        ss_tot = torch.sum((y - torch.mean(y)) ** 2)
        return 1 - (ss_res / ss_tot)

def plot_metrics(self):
    fig, axes = plt.subplots(3, figsize=(18, 5))

    # MSE
    axes[0].plot(self.history.train_loss_history, label='Train')
    if self.history.val_loss_history:
        axes[0].plot(self.history.val_loss_history,
label='Validation')
    axes[0].set_title('MSE during Training')
    axes[0].set_yscale('log')
    axes[0].legend()
    axes[0].grid(True)

    # MAE
    axes[1].plot(self.history.train_mae_history, label='Train')
    if self.history.val_mae_history:
        axes[1].plot(self.history.val_mae_history, label='Validation')
    axes[1].set_title('MAE during Training')
    axes[1].set_yscale('log')
    axes[1].legend()
    axes[1].grid(True)

    # RMSE
    axes[2].plot(self.history.train_rmse_history, label='Train')
    if self.history.val_rmse_history:
        axes[2].plot(self.history.val_rmse_history,
label='Validation')
    axes[2].set_title('RMSE during Training')
    axes[2].set_yscale('log')
    axes[2].legend()
    axes[2].grid(True)

    plt.tight_layout()
    plt.show()

```

Листинг 2 - Реализация логистической регрессии с регуляризациями

```
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score, accuracy_score
import torch

@dataclass
class TrainingHistoryLogistic:
    losses: list[float]
    accuracies: list[float]
    f1_scores: list[float]
    val_losses: list[float]
    val_accuracies: list[float]
    val_f1_scores: list[float]
    weights_norm: list[float]

class LogisticRegression:
    def __init__(self, learning_rate: float = 0.01, n_iterations: int = 1000, regularization: str = None,
                 tolerance: float = 1e-6, fit_intercept: bool = True,
                 verbose: bool = False, alpha: float = 0.01, l1_ratio: float = 0.5):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.tolerance = tolerance
        self.fit_intercept = fit_intercept
        self.verbose = verbose
        self.weights = None
        self.bias = 0.0 if fit_intercept else None
        self.is_fitted = False
        self.history = None
        self.regularization = regularization
        self.alpha = alpha
        self.l1_ratio = l1_ratio

    @staticmethod
    def sigmoid(z: torch.Tensor) -> torch.Tensor:
        return torch.sigmoid(z)

    def compute_loss(self, y_true: torch.Tensor, y_pred: torch.Tensor,
                    weights: torch.Tensor) -> torch.Tensor:
        epsilon = 1e-15
        y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
        base_loss = -torch.mean(y_true.to(torch.float32) *
                                torch.log(y_pred) + (1 - y_true.to(torch.float32)) * torch.log(1 -
                                y_pred))
```

```

    reg = 0
    if self.regularization == 'l1':
        reg = self.alpha * torch.sum(torch.abs(weights))
    elif self.regularization == 'l2':
        reg = self.alpha * torch.sum(weights ** 2)
    elif self.regularization == 'elasticnet':
        l1_reg = self.alpha * self.l1_ratio *
torch.sum(torch.abs(weights))
        l2_reg = self.alpha * (1 - self.l1_ratio) *
torch.sum(weights ** 2)
        reg = l1_reg + l2_reg

    return base_loss + reg

def compute_gradients(self, X: torch.Tensor, error: torch.Tensor,
weights: torch.Tensor) -> tuple:
    n_samples = X.shape[0]
    dw = (1 / n_samples) * torch.matmul(X.T, error)

    if self.regularization == 'l1':
        dw += self.alpha * torch.sign(weights)
    elif self.regularization == 'l2':
        dw += 2 * self.alpha * weights
    elif self.regularization == 'elasticnet':
        l1_grad = self.alpha * self.l1_ratio * torch.sign(weights)
        l2_grad = 2 * self.alpha * (1 - self.l1_ratio) * weights
        dw += l1_grad + l2_grad

    db = (1 / n_samples) * torch.sum(error) if self.fit_intercept
else 0

    return dw, db

def gradient_descent(self, X: torch.Tensor, y: torch.Tensor, X_val:
torch.Tensor = None, y_val: torch.Tensor = None):
    n_samples, n_features = X.shape
    self.weights = torch.zeros(n_features, device=device)
    if self.fit_intercept:
        self.bias = torch.tensor(0.0, device=device)

    losses, accuracies, f1_scores = [], [], []
    val_losses, val_accuracies, val_f1_scores, weights_norm = [],
[], [], []

    for iteration in range(self.n_iterations):

```

```

        linear_model = torch.matmul(X, self.weights)
        if self.fit_intercept:
            linear_model += self.bias
        y_pred_prob = self.sigmoid(linear_model)
        y_pred = (y_pred_prob >= 0.5).to(torch.int)

        loss = self.compute_loss(y, y_pred_prob, self.weights)
        acc = accuracy_score(y.detach().cpu().numpy(),
y_pred.detach().cpu().numpy())
        f1 = f1_score(y.detach().cpu().numpy(),
y_pred.detach().cpu().numpy())

        losses.append(loss.item())
        accuracies.append(acc)
        f1_scores.append(f1)
        weights_norm.append(torch.norm(self.weights).item())

    # Validation metrics
    if X_val is not None and y_val is not None:
        val_prob = self.sigmoid(torch.matmul(X_val,
self.weights) + (self.bias if self.fit_intercept else 0))
        val_pred = (val_prob >= 0.5).to(torch.int)
        val_loss = self.compute_loss(y_val, val_prob,
self.weights)
        val_acc = accuracy_score(y_val.detach().cpu().numpy(),
val_pred.detach().cpu().numpy())
        val_f1 = f1_score(y_val.detach().cpu().numpy(),
val_pred.detach().cpu().numpy())

        val_losses.append(val_loss.item())
        val_accuracies.append(val_acc)
        val_f1_scores.append(val_f1)

    # Gradients
    error = y_pred_prob - y
    dw, db = self.compute_gradients(X, error, self.weights)

    self.weights -= self.learning_rate * dw
    if self.fit_intercept:
        self.bias -= self.learning_rate * db

    # if iteration > 0 and abs(losses[-2] - losses[-1]) <
self.tolerance:
    #     if self.verbose:
    #         print(f"Converged at iteration {iteration}")
    #     break

```

```

        if self.verbose and iteration % 100 == 0:
            msg = f"Iteration {iteration}, Train Loss: {loss:.6f},
Acc: {acc:.3f}, F1: {f1:.3f}"
            if X_val is not None:
                msg += f", Val Loss: {val_loss:.6f}, Val Acc:
{val_acc:.3f}, Val F1: {val_f1:.3f}"
            print(msg)

    self.history = TrainingHistoryLogistic(
        losses=losses,
        accuracies=accuracies,
        f1_scores=f1_scores,
        val_losses=val_losses,
        val_accuracies=val_accuracies,
        val_f1_scores=val_f1_scores,
        weights_norm=weights_norm
    )

    def fit(self, X: torch.Tensor, y: torch.Tensor, X_val: torch.Tensor =
None, y_val: torch.Tensor = None):
        self.gradient_descent(X, y, X_val, y_val)
        self.is_fitted = True

    def predict_proba(self, X: torch.Tensor) -> torch.Tensor:
        linear_model = torch.matmul(X, self.weights)
        if self.fit_intercept:
            linear_model += self.bias
        return self.sigmoid(linear_model)

    def predict(self, X: torch.Tensor, threshold: float = 0.5) ->
torch.Tensor:
        proba = self.predict_proba(X)
        return (proba >= threshold).float()

    def score(self, X: torch.Tensor, y: torch.Tensor) -> float:
        y_pred = self.predict(X)
        return (y_pred == y).float().mean().item()

    def plot_metrics(self):
        fig, axes = plt.subplots(3, figsize=(15, 10))

        # Loss
        axes[0].plot(self.history.losses, label='Train', linewidth=2)
        axes[0].plot(self.history.val_losses, label='Validation',
linewidth=2)

```

```

axes[0].set_title("Loss")
axes[0].set_xlabel("Iterations")
axes[0].set_ylabel("Loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Accuracy
axes[1].plot(self.history accuracies, label='Train',
linewidth=2)
axes[1].plot(self.history.val accuracies, label='Validation',
linewidth=2)
axes[1].set_title("Accuracy")
axes[1].set_xlabel("Iterations")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# F1 Score
axes[2].plot(self.history.f1_scores, label='Train', linewidth=2)
axes[2].plot(self.history.val_f1_scores, label='Validation',
linewidth=2)
axes[2].set_title("F1 Score")
axes[2].set_xlabel("Iterations")
axes[2].set_ylabel("F1 Score")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.suptitle(f"Logistic Regression Training - Regularization:
{self.regularization or 'None'}",
            fontsize=16)
plt.tight_layout()
plt.show()

```

Листинг 3 - Применение линейной регрессии с регуляризациями

```
target_column = 'Median_House_Value'
X_cleaned = californication.drop(target_column,
axis=1).values.astype(np.float32)
y_cleaned = californication[target_column].values.astype(np.float32)

separator = DataSeparator(test_size=0.15, val_size=0.15, shuffle=True,
seed=42)
data_splits = separator.separate(X_cleaned, y_cleaned)

scaler = StandardScaler()
scaler.fit(data_splits.X_train)

data_splits.X_train = scaler.transform(data_splits.X_train)
data_splits.X_val = scaler.transform(data_splits.X_val)
data_splits.X_test = scaler.transform(data_splits.X_test)

models= [LinearRegression(learning_rate=0.1, n_iterations=5000,
verbose=True, regularization='l1', alpha=2),
          LinearRegression(learning_rate=0.1, n_iterations=5000,
verbose=True, regularization='l2', alpha=2),
          LinearRegression(learning_rate=0.1, n_iterations=5000,
verbose=True, regularization='elasticnet', alpha=2, l1_ratio=0.5),
          LinearRegression(learning_rate=0.1, n_iterations=5000,
verbose=True, regularization=None)]

for model in models:
    print(f"\nTraining {model.regularization} model")
    model.fit(data_splits.X_train, data_splits.y_train, data_splits.X_val,
data_splits.y_val)

    predictions = model.predict(data_splits.X_test)

    r2_test = model.score(data_splits.X_test, data_splits.y_test)
    r2_train = model.score(data_splits.X_train, data_splits.y_train)

    # model.plot_metrics()

    print("\nРазмеры выборок:")
    print(f"Train: {data_splits.X_train.shape[0]} samples")
    print(f"R^2 на тестовой выборке: {r2_test:.4f}")
    print(f"R^2 на обучающие выборке: {r2_train:.4f}")
    print(f"Validation: {data_splits.X_val.shape[0]} samples")
    print(f"Test: {data_splits.X_test.shape[0]} samples")
    print(f"Features: {data_splits.X_train.shape[1]} features")
```



```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
colors = ['blue', 'red', 'green', 'orange', 'purple', 'brown']
for i in range (len(models)):
    model = models[i]
    if model.history is not None:
        color = colors[i % len(colors)]
        axes[0].plot(model.history.train_loss_history,
                      label=f'{model.regularization} (Train)',
                      color=color,
                      linewidth=2)
        axes[1].plot(model.history.train_mae_history,
                      label=f'{model.regularization} (Train)',
                      color=color,
                      linewidth=2)
        axes[2].plot(model.history.train_rmse_history,
                      label=f'{model.regularization} (Train)',
                      color=color,
                      linewidth=2)

axes[0].set_title('Train MSE Comparison')
axes[0].set_xlabel('Iterations')
axes[0].set_ylabel('MSE')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_title('Train MAE Comparison')
axes[1].set_xlabel('Iterations')
axes[1].set_ylabel('MAE')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

axes[2].set_title('Train RMSE Comparison')
axes[2].set_xlabel('Iterations')
axes[2].set_ylabel('RMSE')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Листинг 4 - Применение логистической регрессии с регуляризациями

```
target_column = 'Survived'
X_cleaned = titanic.drop(target_column,
axis=1).values.astype(np.float32)
y_cleaned = titanic[target_column].values.astype(np.float32)

separator = DataSeparator(test_size=0.15, val_size=0.15, shuffle=True,
seed=42)
data_splits = separator.separate(X_cleaned, y_cleaned)

scaler = StandardScaler()
scaler.fit(data_splits.X_train)

data_splits.X_train = scaler.transform(data_splits.X_train)
data_splits.X_val = scaler.transform(data_splits.X_val)
data_splits.X_test = scaler.transform(data_splits.X_test)

models= [LogisticRegression(learning_rate=0.1, n_iterations=1000,
verbose=True, regularization='l1', alpha=0.1),
         LogisticRegression(learning_rate=0.1, n_iterations=1000,
verbose=True, regularization='l2', alpha=0.1),
         LogisticRegression(learning_rate=0.1, n_iterations=1000,
verbose=True, regularization='elasticnet', alpha=0.1, l1_ratio=0.5),
         LogisticRegression(learning_rate=0.1, n_iterations=1000,
verbose=True, regularization=None)]

for model in models:
    print(f"\nTraining {model.regularization} model")
    model.fit(data_splits.X_train, data_splits.y_train, data_splits.X_val,
data_splits.y_val)

    predictions = model.predict(data_splits.X_test)

    # model.plot_metrics()

    print("Размеры выборок:")
    print(f"Train: {data_splits.X_train.shape[0]} samples")
    print(f"Validation: {data_splits.X_val.shape[0]} samples")
    print(f"Test: {data_splits.X_test.shape[0]} samples")
    print(f"Features: {data_splits.X_train.shape[1]} features")
    print(f"Accuracy на тестовой выборке: {model.score(data_splits.X_test,
data_splits.y_test)}")
    print(f"F1 на тестовой выборке: {f1_score(data_splits.y_test.cpu(),
predictions.cpu())}")
    print(f"Accuracy на обучающей выборке:
```

```

{model.score(data_splits.X_train, data_splits.y_train)}")
    print(f"F1 на обучающей выборке: {f1_score(data_splits.y_train.cpu(),
model.predict(data_splits.X_train).cpu())}")

fig, axes = plt.subplots(1, 2, figsize=(15, 7))
colors = ['blue', 'red', 'green', 'orange', 'purple', 'brown']
for i in range (len(models)):
    model = models[i]
    if model.history is not None:
        color = colors[i % len(colors)]
        axes[0].plot(model.history accuracies,
                      label=f'{model.regularization} (Train)',
                      color=color,
                      linewidth=2)
        axes[1].plot(model.history.losses,
                      label=f'{model.regularization} (Train)',
                      color=color,
                      linewidth=2)
axes[0].set_title('Train Accuracy Comparison')
axes[0].set_xlabel('Iterations')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_title('Train Losses Comparison')
axes[1].set_xlabel('Iterations')
axes[1].set_ylabel('Losses')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Оценка моделей

Далее на Листингах 5-6 представлены результаты работы моделей

Листинг 5 - Результат работы линейной модели

Training l1 model

Размеры выборок:

Train: 14448 samples

R^2 на тестовой выборке: 0.6080

R^2 на обучающие выборке: 0.5862

Validation: 3096 samples

Test: 3096 samples

Features: 6 features

Training l2 model

Размеры выборок:

Train: 14448 samples

R^2 на тестовой выборке: 0.2533

R^2 на обучающие выборке: 0.2445

Validation: 3096 samples

Test: 3096 samples

Features: 6 features

Training elasticnet model

Размеры выборок:

Train: 14448 samples

R^2 на тестовой выборке: 0.3723

R^2 на обучающие выборке: 0.3591

Validation: 3096 samples

Test: 3096 samples

Features: 6 features

Training None model

Размеры выборок:

Train: 14448 samples

R^2 на тестовой выборке: 0.6080

R^2 на обучающие выборке: 0.5862

Validation: 3096 samples

Test: 3096 samples

Features: 6 features

Листинг 6 - Результат работы логистической регрессии

Training l1 model

Размеры выборок:

Train: 623 samples

Validation: 133 samples

Test: 133 samples

Features: 7 features

Accurasy на тестовой выборке: 0.7894737124443054

F1 на тестовой выборке: 0.6888888888888889

Accurasy на обучающей выборке: 0.778491199016571

F1 на обучающей выборке: 0.7025862068965517

Training l2 model

Размеры выборок:

Train: 623 samples

Validation: 133 samples

Test: 133 samples

Features: 7 features

Accurasy на тестовой выборке: 0.8120300769805908

F1 на тестовой выборке: 0.6835443037974683

Accurasy на обучающей выборке: 0.8025681972503662

F1 на обучающей выборке: 0.7092198581560284

Training elasticnet model

Размеры выборок:

Train: 623 samples

Validation: 133 samples

Test: 133 samples

Features: 7 features

Accurasy на тестовой выборке: 0.7894737124443054

F1 на тестовой выборке: 0.6888888888888889

Accurasy на обучающей выборке: 0.778491199016571

F1 на обучающей выборке: 0.7025862068965517

Training None model

Размеры выборок:

Train: 623 samples

Validation: 133 samples

Test: 133 samples

Features: 7 features

Accurasy на тестовой выборке: 0.8120300769805908

F1 на тестовой выборке: 0.7311827956989247
Аккуратность на обучающей выборке: 0.7720706462860107
F1 на обучающей выборке: 0.7066115702479339

Далее на рисунках 1-2 представлены графики изменения ошибок и метрик каждой модели с регуляризациями и без.

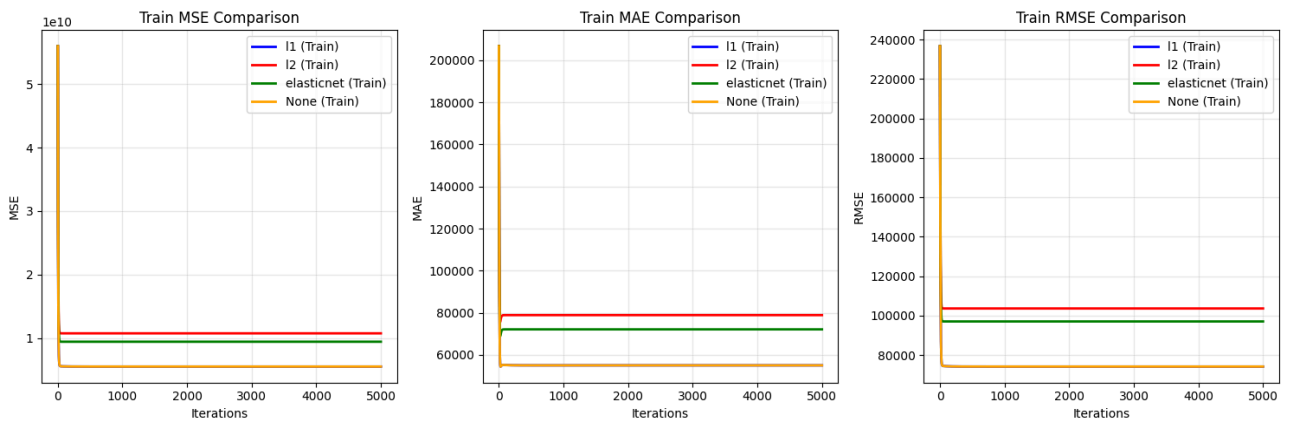


Рисунок 1 - Графики изменения MSE, MAE и RMSE

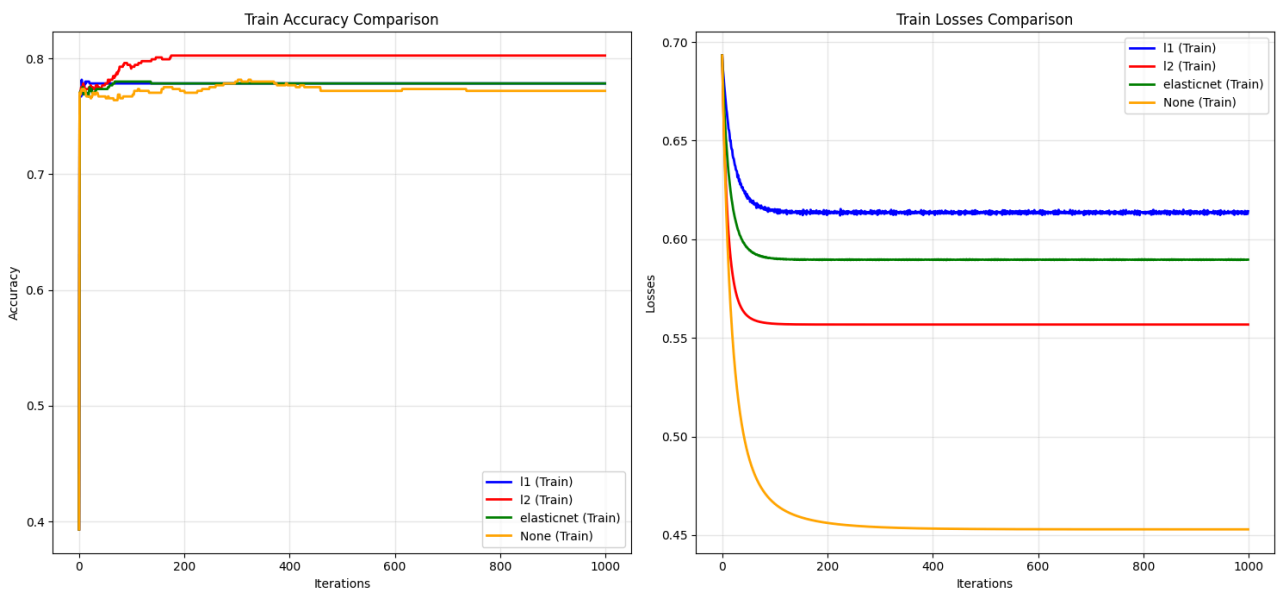


Рисунок 2 - Графики изменения Аккуратность и Losses

Часть 2

Алгоритмы

Ниже представлен псевдокод для алгоритма К-ближайших соседей (KNN).

a) Для каждого объекта x_{train} из X_{train} :

 Вычислить расстояние $d = \text{dist}(x_{\text{test}}, x_{\text{train}})$

b) Составить список пар (d, y_{train})

c) Отсортировать список по расстоянию d в порядке возрастания

d) Выбрать k ближайших соседей:

$\text{neighbors} =$ первые k элементов списка

e) Если задача классификации:

 Определить метку y_{pred} как наиболее частую среди меток соседей

 Иначе, если задача регрессии:

 Определить y_{pred} как среднее значение меток соседей

f) Добавить y_{pred} в список предсказаний

Реализация

Далее на Листингах 7 - 9 представлены реализация модели на основе алгоритма KNN и её применение на задаче классификации и регрессии.

Листинг 7 - Реализация модели на основе алгоритма KNN.

```
import numpy as np
import torch

class KNN:
    def __init__(self, k: int = 5, distance: str = None, task: str = None,
batch_size: int = 500, device=None) -> None:
        self.k = k
        self.distance = distance
        self.task = task
        self.is_fitted = False
        self.X_train: torch.Tensor = None
        self.y_train: torch.Tensor = None
        self.batch_size = batch_size
        self.device = device or ('cuda' if torch.cuda.is_available() else
'cpu')

    def _compute_distance(self, X: torch.Tensor) -> torch.Tensor:
        X = X.to(self.device)
        X_train = self.X_train

        if self.distance == "euclidean":
            X2 = torch.sum(X**2, dim=1, keepdims=True)
            T2 = torch.sum(self.X_train**2, dim=1).unsqueeze(0)
            D2 = X2 + T2 - 2 * X @ self.X_train.T
            dists = torch.sqrt(torch.clamp(D2, min=0.0))
        elif self.distance == "manhattan":
            dists = torch.cdist(X, X_train, p=1)
        elif self.distance == "cos":
            dists = 1 - torch.mm(
                torch.nn.functional.normalize(X, dim=1),
                torch.nn.functional.normalize(X_train, dim=1).T
            )
        else:
            raise ValueError(f"Я не знаю че это такое: {self.distance}")

        return dists
```



```

def predict(self, X: torch.Tensor) -> torch.Tensor:
    if self.task is None:
        raise ValueError("Выберите задачу :( я откуда знаю, что вам нужно?")
    if self.distance is None:
        raise ValueError("Выберите какое расстояние считать :(")
    if self.is_fitted:
        print("Окееей летс гоу")

    print("-----")
    print("-----")

    y_pred = []

    X = X.to(self.device)
    n_samples = X.shape[0]
    for start in range(0, n_samples, self.batch_size):
        end = min(start + self.batch_size, n_samples)
        X_batch = X[start:end]

        print(f"Считаю расстояния для батча {start}-{end}...")
        dists = self._compute_distance(X_batch)
        print("Посчитал!")
        nearest = torch.argsort(dists, axis=1)[:self.k]
        y_neighbors = self.y_train[nearest]

        if self.task == "classification":
            preds = torch.mode(y_neighbors, dim=1).values
        elif self.task == "regression":
            preds = torch.mean(y_neighbors.float(), dim=1)
        else:
            raise ValueError(f"Я не понимаю что вы от меня хотите: {self.task}")

        y_pred.append(preds.detach().cpu())

    y_pred = torch.cat(y_pred)

    print("-----")
    print("-----")

    print("Готово!\n")
    return y_pred

else:
    raise ValueError("Модель еще не стройная🤖")

```

```
def fit(self, X: torch.Tensor, y: torch.Tensor) -> None:
    self.X_train = X.to(self.device)
    self.y_train = y.to(self.device)
    self.is_fitted = True
```

Листинг 8 - Применение модели для задачи классификации.

```
from sklearn.metrics import accuracy_score, classification_report

target_column = "label"
X_train = torch.tensor(fashion_train.drop(target_column, axis=1).values,
dtype=torch.float32)
y_train = torch.tensor(fashion_train[target_column].values)
X_test = torch.tensor(fashion_test.drop(target_column, axis=1).values,
dtype=torch.float32)
y_test = torch.tensor(fashion_test[target_column].values,
dtype=torch.long)

model = KNN(k = 15, distance="manhattan", task="classification",
batch_size = 500, device=device)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Аккураси:", accuracy_score(y_test.cpu().numpy(),
y_pred.cpu().numpy()))
print(classification_report(y_test.cpu().numpy(), y_pred.cpu().numpy()))
```

Листинг 9 - Применение модели для задачи регрессии.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

target_column = 'charges'
X_cleaned = insurance.drop(target_column,
axis=1).values.astype(np.float32)
y_cleaned = insurance[target_column].values.astype(np.float32)

separator = DataSeparator(test_size=0.15, val_size=0.15, shuffle=True,
seed=42)
data_splits = separator.separate(X_cleaned, y_cleaned)

scaler = StandardScaler()
scaler.fit(data_splits.X_train)
```

```

data_splits.X_train = scaler.transform(data_splits.X_train)
data_splits.X_val = scaler.transform(data_splits.X_val)
data_splits.X_test = scaler.transform(data_splits.X_test)

model = KNN(k = 13, distance="cos", task="regression", batch_size=50,
device=device)
model.fit(data_splits.X_train, data_splits.y_train)

y_pred = model.predict(data_splits.X_test)

mae = mean_absolute_error(data_splits.y_test.cpu().numpy(),
y_pred.cpu().numpy())
mse = mean_squared_error(data_splits.y_test.cpu().numpy(),
y_pred.cpu().numpy())
rmse = np.sqrt(mse)
r2 = r2_score(data_splits.y_test.cpu().numpy(), y_pred.cpu().numpy())

print(f"MAE: {mae:.3f}")
print(f"MSE: {mse:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"R²: {r2:.3f}")

```

Оценка моделей

Далее на Листингах 10 - 11 представлены результаты работы модели для задач классификации и регрессии.

Листинг 10 - Результат работы на задаче классификации.

Аккураси: 0.8604				
	precision	recall	f1-score	support
0	0.76	0.88	0.82	1000
1	0.99	0.96	0.97	1000
2	0.76	0.79	0.77	1000
3	0.92	0.89	0.90	1000
4	0.78	0.82	0.80	1000
5	0.99	0.86	0.92	1000
6	0.67	0.57	0.61	1000
7	0.88	0.94	0.91	1000
8	0.98	0.94	0.96	1000
9	0.90	0.97	0.93	1000
accuracy			0.86	10000
macro avg	0.86	0.86	0.86	10000
weighted avg	0.86	0.86	0.86	10000

Листинг 11 - Результат работы на задаче регрессии.

MAE: 3780.461
MSE: 34908944.000
RMSE: 5908.379
R ² : 0.747

Вывод

В процессе выполнения лабораторной работы я познакомилась с такими регуляризациями как L1, L2 и ElasticNet, использовала их при обучении моделей, опробовала их на реальных данных, пронаблюдала прогресс их обучения за счёт снятия метрик. Также я познакомилась с алгоритмом обучения KNN и применила его на реальных данных.

Приложение

Исходный код лабораторной работы представлен по ссылке:
https://colab.research.google.com/drive/1reznMJhspzoO8cpJdvo4I_G1SN109muT?usp=sharing